# Global Congestion-Balanced Slotting Optimization

Exact layout assignment for smoother warehouse traffic and lower operational downtime

Layout Management System Proposal

27 July 2026

# Decision requested

## Approve exact congestion-balanced placement

Assign complete movable units across the whole warehouse rather than relying only on pair swaps.

- Globally minimize the worst movement bottleneck
- Prevent popular racks from clustering in one zone or aisle
- Preserve every existing storage hard rule
- Constrain additional travel and report the optimality gap

PROPOSED OUTCOME

**Smoother flow**

Lower peak utilization and queue concentration.

OPTIMIZATION CLAIM

**OPTIMAL**

Global optimum is claimed only at a 0% gap.

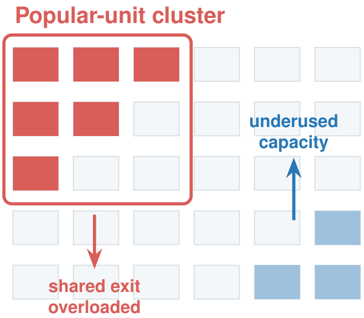
# Problem to solve

## Current operational risk

Popular shelves or totes can accumulate in the same zone, aisle, lift, or crane service area.

This concentrates traffic even when total warehouse capacity is adequate.

- Queueing at shared entrances and transfer points
- Uneven robot or crane workload
- Longer waiting time and lower throughput



# Success definition

PRIMARY

**Peak**

Minimize the maximum normalized load across lanes, lifts, cranes, intersections, and transfer points.

SPATIAL BALANCE

**Local**

Prevent high-visit movable units from accumulating in one compatible zone or shared-resource neighbourhood.

SERVICE QUALITY

**P95**

Reduce the high-load tail so traffic is smooth across more than a single worst bottleneck.

GUARDRAIL

**Travel**

Keep total expected travel within a configured increase from the starting layout.

**BALANCE BY CAPACITY, NOT RAW VISITS**

**ALL HARD RULES REMAIN ENFORCED**

# Current method versus proposed method

## Current: greedy pair swaps

- Starts from an existing, ABC, or affinity layout
- Exchanges two complete handling units
- Accepts immediate lexicographic improvements
- Stops after a limited number of iterations

FAST LOCAL OPTIMUM RISK

## Proposed: exact global assignment

- Assigns every movable unit to a compatible location
- Evaluates resource, zone, and local concentration together
- Uses branch-and-bound to prove the best model solution
- Reports incumbent, lower bound, and optimality gap

GLOBALLY AUDITABLE TIME-BOUNDED OPTION

# Universal movable-unit abstraction

**Store ID + Date task demand** → **Movable unit visit frequency** → **Compatible candidate locations** → **Route-resource contribution**

**AMR**
Movable unit = complete shelf. Related SKUs remain on the shelf; popular shelves are distributed across compatible rack positions.

**ASRS**
Movable unit = tote, pallet, tray, or multi-slot item group. Demand is distributed across compatible aisles, cranes, or lifts.

One optimization interface; system-specific units and resource networks.

# Demand and traffic model

HISTORICAL DEMAND

$$v_u = \sum_{t \in \mathcal{T}} \mathbf{1}\{\text{task } t \text{ requires unit } u\}$$

- One task is one Store ID + Date
- AMR shelf is counted once per task
- ASRS counts each required physical retrieval

PRECOMPUTED ROUTE CONTRIBUTION

$$a_{ulr} = v_u \times \mathbf{1}\{\text{route from } l \text{ uses } r\}$$

- $u$: movable unit
- $l$: candidate storage location
- $r$: shared movement resource

## Fixed-route scope

Shortest routes are precomputed for every candidate location. This makes layout assignment exact and tractable while matching the current expected-flow model.

# Exact placement model

DECISION VARIABLE

$$x_{ul} = \begin{cases} 1 & \text{unit } u \text{ is assigned to location } l \\ 0 & \text{otherwise} \end{cases}$$

ONE LOCATION PER UNIT

$$\sum_{l \in L(u)} x_{ul} = 1 \qquad \forall u$$

LOCATION OCCUPANCY

$$\sum_{u:\, l \in L(u)} x_{ul} \leq 1 \qquad \forall l$$

RESOURCE UTILIZATION

$$q_r = \frac{1}{C_r} \sum_{u} \sum_{l \in L(u)} a_{ulr} x_{ul}$$

$L(u)$ contains only hard-rule-compatible locations; invalid placement variables are never created.

# Congestion must be measured at three levels

## 1. Shared resource

Lane, aisle entrance, lift, crane, intersection, conveyor, or transfer point.

$$Q^R = \max_r q_r$$

## 2. Zone

Workload normalized by the compatible zone's service capacity.

$$Q^Z = \max_z \frac{\sum_{u,l \in z} v_u x_{ul}}{C_z}$$

## 3. Local neighbourhood

Racks sharing a physical entrance, lift, crane, or defined grid window.

$$Q^N = \max_n \frac{\sum_{u,l \in n} v_u x_{ul}}{C_n}$$

**RECOMMENDATION** Define neighbourhoods from shared physical resources before using arbitrary grid windows.

# Lexicographic global objective

- **Stage 1** **Minimize maximum movement-resource utilization** $Q^R$
- **Stage 2** **Fix Stage 1 optimum; minimize local concentration** $Q^N$
- **Stage 3** **Fix earlier optima; minimize zone utilization** $Q^Z$
- **Stage 4** **Minimize P95/CVaR high-load tail across resources**
- **Stage 5** **Minimize travel, then relocation count**

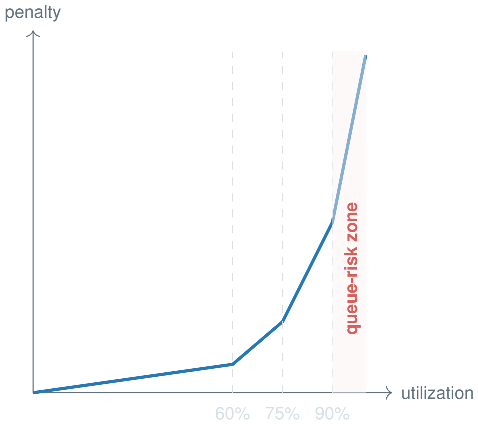
Earlier objectives cannot be sacrificed for a later convenience metric.

# Queue-risk penalty for smoother traffic

Average load is not enough: waiting time rises sharply as utilization approaches capacity.

Use a convex piecewise-linear penalty:

| Utilization | Marginal penalty |
|---|---|
| 0–60% | Low |
| 60–75% | Moderate |
| 75–90% | High |
| $>$ 90% | Extreme |




**Effect:** spread traffic before a bottleneck reaches unstable utilization.

# Hard rules remain non-negotiable

- Chilled inventory remains in chilled-compatible storage
- Ambient standard and oversize segments remain non-mixed
- Dimensions, rotation, weight, and contiguous footprint must fit
- Known overweight inventory remains on its required ergonomic level
- Custom inherited zone/rack/slot attributes remain enforced

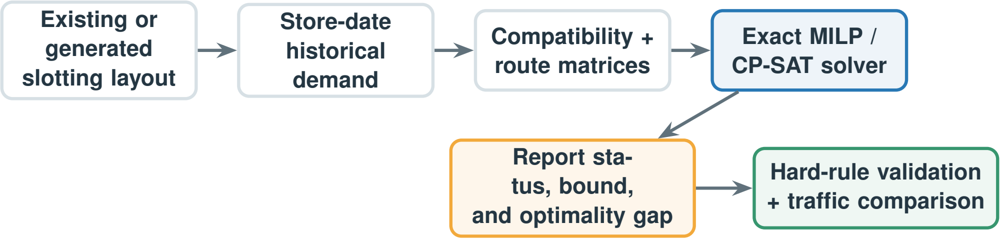
MODEL CONSTRUCTION RULE

If unit $u$ cannot use location $l$, the variable $x_{ul}$ is not created.

UNKNOWN PHYSICAL DATA

Preserve the current conservative policy: keep the unit fixed unless an approved compatible exception location is known.

# End-to-end solver workflow




OPTIMAL: gap = 0%     FEASIBLE: report remaining gap

# Integration with the current application

## Keep both baseline workflows

1. Optimize an existing `.slotting.json`
2. Generate ABC or ABC + affinity, then optimize

Add a solver-mode selector:

**QUICK HEURISTIC** **EXACT / BOUNDED**

## New exact-mode settings

- Solve-time limit
- Maximum travel increase
- Optimality-gap target
- Neighbourhood definition
- Solver backend

Save the model status, bound, gap, objectives, and placement audit in the resulting `.slotting.json`.

# Solver strategy and global-optimum claim

## Exact mode

Continue branch-and-bound until:

$$\text{optimality gap} = 0\%$$

Output status: **OPTIMAL**

Recommended backend:

- Gurobi or CPLEX for strongest MILP performance
- OR-Tools CP-SAT for an open-source pilot

## Time-bounded mode

Stop after a configured duration and report:

$$\text{gap} = \frac{\text{incumbent} - \text{bound}}{|\text{incumbent}|}$$

Output status: **FEASIBLE**

A feasible solution must never be labelled globally optimal.

# Validation: optimization first, simulation second

| Exact expected-flow layout optimization | Discrete-event traffic simulation | Controlled warehouse pilot and monitoring |
|---|---|---|
| Peak/P95 utilization<br>Zone and local balance<br>Travel and relocations | Queue time<br>Throughput<br>Resource waiting<br>Task completion time | Actual downtime<br>Operational exceptions<br>Model calibration |

Exact optimization proves the best layout for the fixed-route model; simulation tests whether that model predicts real operational improvement.

# Delivery plan

| Phase | Deliverable | Estimate | Exit evidence |
|---|---|---|---|
| 1. Data model | Movable-unit, candidate-location, resource, zone, and neighbourhood matrices | 3–4 days | Matrix tests |
| 2. Exact solver | Placement constraints, lexicographic objectives, warm start, bounds, and gap | 5–7 days | Small-instance proof |
| 3. Product integration | Solver mode, progress/cancel, JSON audit, map comparison, and error handling | 3–4 days | GUI workflow test |
| 4. Validation | Benchmark layouts, simulation comparison, performance tuning, and documentation | 4–6 days | Acceptance report |

**TOTAL: 15–21 ENGINEERING DAYS**

**PILOT: APPROXIMATELY 3–4 WEEKS**

# Key risks and controls

## Solver runtime

**Risk:** assignment size grows with units × locations.
**Control:** compatibility pruning, warm starts, symmetry breaking, and explicit exact/time-bounded modes.

## Weak capacity data

**Risk:** relative loads may not represent real bottlenecks.
**Control:** support measured capacities and preserve transparent relative-mode reporting.

## Static-route approximation

**Risk:** controllers may reroute around congestion.
**Control:** validate with simulation and later add scenario or route-choice extensions.

## Operational churn

**Risk:** mathematically strong layouts may relocate too many units.
**Control:** travel cap, relocation minimization, and operator-visible move audit.

# Acceptance criteria

FUNCTIONAL

- Existing-layout and full-pipeline entry paths both work
- AMR and ASRS movable units use the same solver interface
- Every current hard rule is validated before and after solving
- Result JSON records status, bound, gap, objectives, and moves

PERFORMANCE AND QUALITY

- Exact benchmark instances finish with 0% gap
- Time-bounded runs return a valid incumbent and honest gap
- Peak and local concentration do not worsen
- Travel remains inside the configured limit

## Pilot success measure

Simulation or controlled operations show lower queue time and smoother resource utilization without violating storage compatibility or increasing travel beyond policy.

# Recommended approval

Proceed with a fixed-route exact pilot that minimizes resource, local, and zone congestion under all existing hard rules.

Deliver both exact and time-bounded modes. Validate predicted downtime reduction with discrete-event simulation before deployment.

GLOBAL WHEN GAP = 0%

AUDITABLE

OPERATIONALLY CONSTRAINED